UNIVERSIDADE PAULISTA
CIÊNCIA DA COMPUTAÇÃO

GUILHERME VALLETTA DOS SANTOS - G811BD6
HENRIQUE BOCCOLI DE OLIVEIRA - N900415
MATHEUS DIAS LAGARES - G785819

DESENVOLVIMENTO DE UM SISTEMA DE IDENTIFICAÇÃO E AUTENTICAÇÃO BIOMÉTRICA

SÃO PAULO
2026

SUMÁRIO

1. OBJETIVO 3
2. INTRODUÇÃO 4
3. FUNDAMENTOS DAS PRINCIPAIS TÉCNICAS BIOMÉTRICAS 6
3.1. Impressão Digital 6
3.2. Reconhecimento Biométrico Facial 7
3.4. Reconhecimento por Voz 9
4. PLANO DE DESENVOLVIMENTO DA APLICAÇÃO 10
4.1. Definição do Problema e Solução Proposta 10
4.2. Ambiente de Desenvolvimento e Ferramental 11
4.3. Elementos da Interface Gráfica (Frontend) 12
4.4. Elementos de Processamento de Imagem e Biometria 13
4.5. Elementos de Persistência de Dados (Backend) 16
5. PROJETO (ESTRUTURA E MÓDULOS DO PROGRAMA) 17
5.1. Arquitetura de Software: Model-View-Controller (MVC) 17
5.2. Estrutura Modular do Programa 18
5.3. Fluxo de Dados e Casos de Uso 21
6. RELATÓRIO COM AS LINHAS DE CÓDIGO DO PROGRAMA 22
6.1. Definição do Banco de Dados (DatabaseService.java) 22
6.2. Detecção de Rosto (BiometriaService.java) 22
6.3. Comparação Biométrica (BiometriaService.java) 23
6.4. Integração com a Webcam (BiometriaService.java) 24
6.5. Aplicando a Regra de Acesso (Login) 24
7. APRESENTAÇÃO DO PROGRAMA EM FUNCIONAMENTO 26
8. BIBLIOGRAFIA 28
9. FICHA ATIVIDADES PRÁTICAS SUPERVISIONADAS 31

1. OBJETIVO

O objetivo desse trabalho é usar linguagens de programação para criar um sistema de identificação e autenticação biométrica para a segurança de informações estratégicas dentro do Ministério do Meio Ambiente. A base do projeto visa a proteção de um banco de dados que armazena informações sobre propriedades rurais que utilizam agrotóxicos proibidos, o que ocasiona na contaminação de recursos hídricos vitais, como lençois freáticos, rios e mares.

A segurança dessas informações sobre essas práticas ilegais e altamente prejudiciais ao meio ambiente, exige um sistema em que seja praticamente infalível para acessos de pessoas não autorizadas, para evitar manipulações, vazamentos de informações importantes ou destruição de dados. Já que se tais informações fossem acessadas por pessoas não autorizadas, colocariam em risco a integridade do Ministério e a capacidade de fiscalização desses dados, além de colocar em risco a saúde pública e os ecossistemas.

A escolha da biometria como ferramenta de identificação e autenticação não é por acaso, pois esse método oferece um nível de segurança maior em relação a outros métodos de segurança comuns, como as senhas tradicionais. E isso ocorre por a biometria usar características biológicas únicas, o que faz ser impossível desse método ser violado virtualmente. A biometria também garante um acesso rápido e simplificado para as informações por usuários autorizados.

Além da segurança, o programa deve controlar rigorosamente acessos em multiníveis, para que seja garantido que cada usuário acesse apenas o que é necessário para suas funções. O programa deve ter como sua inovação e utilidade, a sua capacidade de gerenciar em três níveis diferentes de informação:

- Nível 1 (Acesso Geral): Informações de conhecimento amplo, disponíveis a todos os usuários cadastrados.
- Nível 2 (Acesso Restrito): Dados críticos e mais detalhados, cujo acesso é limitado exclusivamente aos diretores de divisões do Ministério.

- Nível 3 (Acesso Confidencial): As informações mais sensíveis e de alto escalão, acessíveis somente pelo ministro do meio ambiente.

2. INTRODUÇÃO

Nos últimos 20 anos, houve um avanço tecnológico jamais visto antes na história da humanidade, não é por acaso que chamam a nossa sociedade de a “Sociedade da Informação”, esse avanço nos possibilitou de armazenar milhares de informações em apenas alguns HDs por exemplo, além de permitir o acesso a informações que antes era necessário ir em bibliotecas e buscar em dezenas de livros, jornais, artigos científicos, etc. Agora em apenas alguns cliques conseguimos essas informações e muito mais. Isso por um lado foi realmente uma coisa maravilhosa, mas como o mundo não são só “flores”, esse avanço também permitiu que informações que eram guardadas em móveis com trancas, em salas que eram vigiadas e acessadas somente por pessoas autorizadas, serem acessados também com apenas um clique por pessoas não autorizadas, isso elevou a Segurança de Dados a um patamar de extrema importância estratégica em empresas e organizações.

O uso da biometria na questão de cibersegurança vêm se tornando um caso corriqueiro, já que os sistemas de senhas tradicionais estão cada vez mais se tornando “barreiras” transponíveis e se mostrando sistemas ineficientes conforme o avanço da tecnologia, o que pode ocasionar consequências catastróficas para organizações como o Ministério do Meio Ambiente no caso. O fato da biometria ser um identificador único para cada usuário, logicamente dificulta a quebra desse sistema de segurança, tornando-o quase infalível.

A biometria é um método para identificação de indivíduos por suas características físicas e comportamentais. Na Segurança da Informação esse método permite o melhor gerenciamento de acessos e controle de identidade. Existem diversos tipos de segurança biométrica entre elas, se destacam:

- Impressão digital.

- Reconhecimento de face.
- Identificação pela íris.
- Reconhecimento via retina.
- Reconhecimento de voz.
- Geometria da mão.
- Reconhecimento de assinatura.
- Reconhecimento da digitação.
- Estilo de escrita.

A segurança de um sistema não fica somente em quem pode acessá-lo, mas também no que o usuário pode fazer ou ver depois de ter o acesso. Para proteger essas informações de maneira eficaz, decidimos aplicar um modelo hierárquico de confidencialidade.

O banco de dados do Ministério, neste contexto, possui informações classificadas em três níveis de sensibilidade:

- Nível 1 (Público/Geral): Acessível a todos os usuários, compreendendo dados menos críticos.

- Nível 2 (Restrito/Diretoria): Informações detalhadas e operacionais, restritas aos diretores de divisões, que exigem um grau maior de confidencialidade e responsabilidade.

- Nível 3 (Confidencial/Ministro): Dados de altíssimo escalão, confidenciais e estratégicos, cujo acesso é concedido somente ao ministro do meio ambiente.

Este modelo assegura que o princípio do menor privilégio seja estritamente aplicado, limitando o acesso de cada usuário apenas às informações estritamente necessárias para o desempenho de suas funções.

3. FUNDAMENTOS DAS PRINCIPAIS TÉCNICAS BIOMÉTRICAS

A biometria garante segurança, já que ela tem como base um conjunto de características que é difícil de ser replicado. A escolha de acesso por biometria é necessária para quem valoriza e prioriza a segurança, seja da mesma ou da empresa. Algumas das principais técnicas biométricas são:

3.1. Impressão Digital

Esse método de biometria é um dos mais clássicos para a identificação de pessoas. Consiste basicamente em padrões de sulcos e linhas presentes nas digitais de cada pessoa e analisa padrões específicos, como:

- **Sulcos e cristas:** formações elevadas e rebaixadas que compõem as linhas das digitais.

- **Minúcias:** detalhes específicos, como bifurcações, terminações e ilhas nas linhas das digitais.

- **Pontos de singularidade:** áreas únicas, como deltas e vórtices, que ajudam a diferenciar uma digital da outra.

Principais vantagens da Impressão Digital:

- **Alta precisão:** taxa de erro muito baixa.

- **Baixo custo:** implementação acessível e sem necessidade de equipamentos complexos.

- **Facilidade de uso:** processo rápido e intuitivo, amplamente adotado em smartphones e sistemas bancários.

- **Segurança:** difícil de falsificar devido à exclusividade das digitais.

3.2. Reconhecimento Biométrico Facial

Os avanços de tecnologias como IA (Inteligência Artificial) e Machine Learning, o reconhecimento facial tem se tornando mais presente em nossas vidas, como em smartphones.

- **Reconhecimento facial 2D (bidimensional)**

Esse é o mais comum e acessível. A abordagem captura uma imagem plana do rosto, semelhante a uma fotografia comum e analisa características como a distância entre os olhos, o contorno inferior do rosto e o formato do nariz.

- **Reconhecimento facial 3D (tridimensional)**

O reconhecimento facial 3D utiliza câmeras avançadas para capturar a profundidade e os contornos do rosto. A tecnologia gera um modelo digital do rosto que simula dimensões como altura, largura e profundidade. Diferente de uma foto plana e bidimensional, o método oferece maior precisão na identificação.

- **Reconhecimento facial com infravermelho**

Essa tecnologia utiliza a luz infravermelha para mapear características faciais, especialmente os contornos e os detalhes mais sutis do rosto, como veias e outras características internas da face.

- **Reconhecimento facial em movimento (dinâmico)**

O reconhecimento facial em movimento, também conhecido como dinâmico, permite identificar pessoas em movimento, capturando as imagens do rosto de maneira contínua. Em vez de capturar apenas uma foto estática, a tecnologia registra várias imagens do rosto em sequência.

- **Reconhecimento facial com análise de atributos comportamentais**

Esse método analisa não apenas os traços físicos do rosto, mas também o comportamento facial. A tecnologia avalia expressões, como sorrisos ou caretas, e

padrões de movimento, incluindo a forma como os músculos faciais se movem durante a fala ou em diferentes reações.

3.3. Identificação pela Íris

A identificação pela íris é dita por especialistas na área de segurança como uma das tecnologias mais seguras do mundo. Recentemente, a OpenAI causou polêmica no mundo, inclusive no Brasil por coletar esses dados biométricos da íris para criar a World ID, a ideia disso era que essa identificação seria global e permitiria comprovar que o titular era um ser humano único, o que causaria uma maior segurança digital, mas isso não pode ser concluído por conta da suspensão do aplicativo e do projeto pela *Autoridade Nacional de Proteção de Dados (ANPD)* com a justificativa que isso era uma quebra da privacidade e consentimento, e exigiu que a empresa fosse mais transparente sobre o motivo da coleta desses dados. Esses dados são coletados e funcionam dessa forma:

- Mapeamento Único: A íris possui uma estrutura de fibras, cores e texturas complexas que são únicas para cada indivíduo, funcionando como uma "impressão digital" que não se repete nem mesmo entre gêmeos idênticos.

- Captura de Imagem: Um sensor, geralmente a uma distância de cerca de 30 cm, utiliza luz visível ou infravermelha para tirar uma foto de alta resolução da íris.

- Conversão para Código: A imagem capturada é então convertida em um código matemático ou "assinatura digital" anônimo e criptografado.

- Comparação: Este código é comparado com um registro armazenado em um banco de dados para autenticar o indivíduo.

3.4. Reconhecimento por Voz

A voz também é um elemento identificador de indivíduos, já que se forma a partir das cordas vocais, formato da língua, laringe e da boca, o que faz com que cada indivíduo fale de forma única e tenha diferentes timbres. O reconhecimento biométrico acontecesse dessa forma:

- Sistemas de reconhecimento por voz analisam e comparam essas características com um modelo pré-gravado para confirmar a identidade.
- Existem dois tipos principais de autenticação:

    - Dependente de texto: O usuário fala uma frase específica para validação.
    - Independente de texto: A autenticação ocorre em segundo plano,enquanto o usuário fala normalmente durante uma interação.

Esse método é considerado 4 vezes mais rápido que os outros métodos de segurança por biometria, mas tem suas dificuldades para ser aplicado, como:

- **Ruído:** Barulhos de fundo, como outros sons ou vozes, podem interferir na precisão da análise.

- **Fatores comportamentais:** Condições como problemas respiratórios ou tabagismo podem alterar a voz temporariamente.

- **Alterações ao longo do tempo:** A voz pode mudar com a idade, exigindo reconfiguração do sistema.

4. PLANO DE DESENVOLVIMENTO DA APLICAÇÃO

Neste projeto tivemos que escolher elementos e ferramentas para o desenvolvimento. Neste capítulo, mostraremos as ferramentas utilizadas e explicaremos suas funcionalidades e o motivo de sua utilização.

4.1. Definição do Problema e Solução Proposta

A proposta de trabalho exige o desenvolvimento de uma ferramenta em Java (ou C#) capaz de realizar a identificação e autenticação biométrica. O objetivo central é restringir o acesso a um banco de dados sigiloso, simulando informações estratégicas do Ministério do Meio Ambiente.

A proposta define uma hierarquia de acesso:

- **Nível 1 (Público):** Acesso irrestrito.
- **Nível 2 (Restrito):** Acesso permitido a diretores de divisões.
- **Nível 3 (Secreto):** Acesso exclusivo do ministro.

Para atender a essa demanda, a solução proposta consiste em uma aplicação desenvolvida integralmente na plataforma Java. O sistema implementa um fluxo de navegação moderno, permitindo o cadastro de novos usuários e a subsequente verificação de sua identidade através de dois métodos biométricos distintos: reconhecimento de impressão digital (via upload de arquivo) e reconhecimento facial (via captura da webcam).

Uma vez autenticado, o sistema consulta o nível de acesso do usuário e apresenta dinamicamente apenas os dados aos quais ele tem permissão, garantindo a confidencialidade das informações estratégicas.

4.2. Ambiente de Desenvolvimento e Ferramental

A escolha do ambiente de desenvolvimento é fundamental para a produtividade e a qualidade do projeto.

- **Linguagem de Programação: Java (JDK 25)** A linguagem Java foi escolhida por ser um dos requisitos do projeto e por seus méritos técnicos. Sendo uma linguagem Orientada a Objetos, ela facilita a criação de uma arquitetura modular e de fácil manutenção, como a implementada neste projeto (ver Capítulo 7). A sua portabilidade, garantida pela Máquina Virtual Java (JVM), permite que a aplicação final seja executada em diversos sistemas operacionais sem modificação do código-fonte. Utilizamos a versão 25 do JDK (Java Development Kit), uma das mais recentes, para garantir acesso aos recursos de linguagem e APIs modernas.

- **Ambiente de Desenvolvimento Integrado (IDE): IntelliJ IDEA** Para a escrita e depuração do código, optou-se pelo IntelliJ IDEA. Esta IDE oferece um conjunto superior de ferramentas para o desenvolvimento em Java, destacando-se a análise de código estática, a refatoração inteligente e uma integração nativa e fluida com o sistema de build Apache Maven.

- **Gerenciador de Dependências: Apache Maven** O projeto foi estruturado como um projeto Maven. Esta ferramenta é essencial para o gerenciamento do ciclo de vida e das dependências do software. Toda a configuração de bibliotecas externas é centralizada no arquivo pom.xml. Ao invés de baixar e adicionar arquivos JAR manualmente, o Maven automatiza o download e a inclusão das versões corretas das bibliotecas, garantindo a consistência e a reprodutibilidade do projeto. As principais dependências gerenciadas pelo Maven neste projeto foram:
    - org.openjfx: O conjunto de bibliotecas do JavaFX para a interface gráfica.
    - org.bytedeco:javacv-platform: O JavaCV, "ponte" para a biblioteca OpenCV.
    - org.xerial:sqlite-jdbc: O driver JDBC para o banco de dados SQLite.

4.3. Elementos da Interface Gráfica (Frontend)

A interface gráfica (GUI) é o principal ponto de interação do usuário com o sistema. Ela foi construída utilizando a plataforma JavaFX, o framework moderno para aplicações ricas em Java.

- **Framework de UI: JavaFX** O JavaFX foi escolhido em detrimento de tecnologias mais antigas, como o Swing, por sua arquitetura moderna que separa claramente a lógica da apresentação. Ele oferece um conjunto mais rico de componentes, melhor suporte a CSS para estilização e uma integração nativa com elementos multimídia, o que foi crucial para a exibição do stream da webcam.

- **Linguagem de Marcação: FXML** Seguindo as melhores práticas do JavaFX, todas as telas do sistema foram definidas em arquivos FXML (.fxml). O FXML é uma linguagem baseada em XML que permite declarar a estrutura da interface, seus componentes e seus atributos de forma hierárquica e legível. Isso desacopla o design (a "View") da lógica de interação (o "Controller"), permitindo que um designer possa alterar o layout da tela sem necessidade de modificar o código-fonte Java.

- **Componentes Visuais Chave** O sistema utiliza um conjunto de componentes JavaFX para construir suas telas:
    - **Layouts (BorderPane e VBox):** A estrutura principal da aplicação (TelaPrincipal.fxml) é um BorderPane, que atua como uma "concha" mestra. As telas de conteúdo (HomeView, CadastroView, etc.) são carregadas dinamicamente em seu centro. Dentro dessas telas, o VBox é extensivamente utilizado para organizar os formulários verticalmente e, com a propriedade alignment="CENTER", garantir que o conteúdo esteja sempre centralizado na janela.
    - **Controles de Formulário (ComboBox, TextField):** Para a entrada de dados, TextField é usado para campos de texto livre (como nome e usuário), enquanto ComboBox é usado para seleções restritas (Nível de Acesso e Tipo de Biometria), uma escolha de design que previne a entrada de dados inválidos.

- **Multimídia (ImageView):** Este componente é vital para a funcionalidade de reconhecimento facial. Ele é usado na tela de cadastro e verificação para renderizar em tempo real o stream de vídeo capturado pela webcam.
- **Exibição de Dados (TableView):** Na tela de dados sigilosos (DadosView.fxml), o TableView é utilizado para exibir as informações das propriedades rurais de forma estruturada, com colunas e linhas claras.
- **Feedback (Alert):** A classe Alert é usada através de uma classe utilitária (AlertUtils) para fornecer feedback imediato e claro ao usuário, seja em caso de sucesso (ex: "Usuário cadastrado com sucesso!") ou de erro (ex: "Acesso Negado. A biometria não confere.").

4.4. Elementos de Processamento de Imagem e Biometria

Este é o núcleo técnico do projeto. A disciplina de Processamento de Imagem e Visão Computacional (PIVC) é a base para esta seção. A implementação foi realizada utilizando a biblioteca OpenCV, acessada através do wrapper JavaCV.

OpenCV e JavaCV: OpenCV (Open Source Computer Vision Library) é a biblioteca de-facto para tarefas de visão computacional. Por ser escrita em C++ nativo, ela oferece altíssimo desempenho. Para utilizá-la no ambiente Java, empregamos a biblioteca JavaCV (org.bytedeco:javacv-platform), que serve como uma "ponte" (wrapper), expondo as funções nativas do OpenCV para o nosso código Java.

Aquisição de Imagem: O projeto cumpre as orientações da APS ao implementar duas formas de aquisição de imagem:

- **Aquisição por Arquivo (FileChooser):** Utilizado para o cadastro da impressão digital. O FileChooser do JavaFX permite ao usuário selecionar um arquivo de imagem. O caminho desse arquivo é então passado para a função imread do OpenCV, que carrega a imagem na memória como um objeto Mat (Matriz) para processamento.
- **Aquisição por Câmera (OpenCVFrameGrabber):** Utilizado para o reconhecimento facial. O OpenCVFrameGrabber do JavaCV é instanciado para se conectar à webcam padrão. Em uma Thread separada (para não travar a

interface), ele captura quadros de vídeo, que são convertidos em imagens JavaFX (Image) para exibição no ImageView e também salvos como objetos Mat para processamento.

Fundamentos Teóricos da Biometria Implementada: Um conceito-chave do sistema é que ele **não armazena as imagens** da digital ou do rosto. Isso seria uma falha de segurança e ineficiente. Em vez disso, ele armazena um **"template biométrico"**: uma assinatura matemática extraída da imagem.

- **Detecção de Rosto: Haar Cascades** Antes de extrair recursos de um rosto, é preciso encontrá-lo. Para isso, utilizamos um Classificador em Cascata (Cascade Classifier) pré-treinado pelo OpenCV, especificamente o haarcascade_frontalface_alt.xml. Este modelo é treinado com milhares de imagens positivas (rostos) e negativas (não-rostos). Ele funciona deslizando uma janela sobre a imagem em várias escalas e, para cada janela, aplica uma série de "features" (características de Haar) que verificam padrões de luz e sombra (ex: a região dos olhos é mais escura que a testa e as bochechas). Se uma região passa em todas as "etapas" da cascata, ela é classificada como um rosto. No nosso código, o BiometriaService usa esse classificador para obter as coordenadas (Rect) do rosto, recortá-lo e normalizá-lo (redimensionando para 150x150 pixels) antes da extração de recursos.
- **Extração de Recursos: ORB (Oriented FAST and Rotated BRIEF)** Uma vez que temos a imagem biométrica (seja a digital inteira ou o rosto recortado), precisamos criar sua assinatura. Para isso, escolhemos o algoritmo ORB. O ORB é uma fusão eficiente de dois outros algoritmos:
  - **FAST (Features from Accelerated Segment Test):** Um detector de "pontos-chave" (Keypoints) extremamente rápido que encontra cantos e pontos de alta intensidade na imagem (como as minúcias de uma digital ou os cantos dos olhos e boca).
  - **BRIEF (Binary Robust Independent Elementary Features):** Um "descritor" que analisa os pixels ao redor de cada ponto-chave e gera uma string binária (um vetor de 0s e 1s) que descreve sua aparência. O ORB melhora o BRIEF adicionando orientação e resistência à rotação. O resultado final é um objeto Mat contendo centenas de descritores binários. Este Mat é o nosso template

biométrico: rápido de calcular, eficiente para armazenar (poucos kilobytes) e robusto para comparação.

- **Comparação Biométrica: BFMatcher e Teste de Razão (Lowe's Ratio Test)** Quando o usuário tenta se autenticar, o sistema extrai os descritores da nova imagem e precisa compará-los com os descritores armazenados no banco.
  - **BFMatcher (Brute-Force Matcher):** Utilizamos um comparador de força bruta. Como nossos descritores ORB são binários, usamos a NORM_HAMMING (Distância de Hamming), que é uma métrica ultrarrápida para calcular a "diferença" (quantos bits são diferentes) entre dois descritores.
  - **knnMatch (k=2):** Uma simples correspondência 1-para-1 é propensa a falsos positivos. Por isso, usamos o knnMatch para encontrar, para cada descritor da imagem A, os *dois* descritores mais próximos na imagem B.
  - **Lowe's Ratio Test (Teste de Razão de Lowe):** Este é o filtro de qualidade. Iteramos sobre os pares de correspondências. Se a "distância" (custo) da melhor correspondência (M) for significativamente melhor (no nosso caso, 75% ou RATIO_THRESH = 0.75f) que a distância da segunda melhor correspondência (N), consideramos esta uma "boa correlação". Isso elimina correspondências ambíguas.
  - **Limiar Final:** Por fim, contamos o número de "boas correlações". Se esse número for maior que um limiar mínimo (definido como LIMIAR_MINIMO_PONTOS = 15), o sistema conclui que as duas biometrias pertencem à mesma pessoa e concede o acesso.

4.5. Elementos de Persistência de Dados (Backend)

Para armazenar os usuários, seus níveis de acesso e seus templates biométricos, foi implementado um sistema de banco de dados local.

Sistema Gerenciador de Banco de Dados: SQLite A escolha pelo SQLite foi baseada em sua simplicidade e portabilidade. Ele é um banco de dados *serverless*

(sem servidor), o que significa que não requer um serviço separado em execução. O banco de dados inteiro é armazenado em um único arquivo (biometria.db) na raiz do projeto. Isso torna a aplicação autocontida e fácil de distribuir e apresentar.

Driver de Conexão: sqlite-jdbc Para que o Java possa se comunicar com o arquivo SQLite, utilizamos o driver JDBC oficial (org.xerial:sqlite-jdbc), que implementa a API java.sql para conexão e execução de comandos SQL.

Armazenamento de Templates Biométricos: BLOB e Metadados O desafio de armazenar os templates Mat do OpenCV é que eles são objetos de dados binários complexos. A solução foi:

- **Conversão para byte[]:** Criamos métodos utilitários (converterMatParaBytes) que extraem os dados brutos do Mat para um simples array de bytes (byte[]).
- **Armazenamento BLOB:** Este array de bytes é salvo no banco de dados em colunas do tipo BLOB (Binary Large Object), como biometria_dados e face_dados.
- **Armazenamento de Metadados:** Um Mat não é definido apenas por seus dados; ele tem uma estrutura (linhas, colunas, tipo). Para reconstruir o Mat corretamente durante a verificação, também armazenamos suas propriedades: biometria_rows, biometria_cols, e biometria_type.
- **Reconstrução:** Na verificação, o UsuarioDAO lê o BLOB e os metadados, e o BiometriaService usa o método converterBytesParaMat para reconstruir o objeto Mat idêntico ao original.

Estrutura de Dados Sigilosos Para simular o banco de dados do Ministério, criamos a tabela propriedades_rurais. Cada registro contém dados fictícios sobre propriedades, agrotóxicos e, crucialmente, a coluna nivel_acesso_necessario, que permite ao DAO filtrar os resultados com base na permissão do usuário autenticado.

5. PROJETO (ESTRUTURA E MÓDULOS DO PROGRAMA)

Este capítulo descreve a arquitetura de software do sistema, ou seja, a organização e os detalhes de como o código foi estruturado em módulos (pacotes) e como esses módulos interagem para executar as funcionalidades da aplicação.

5.1. Arquitetura de Software: Model-View-Controller (MVC)

O projeto foi desenvolvido seguindo uma variação do padrão de arquitetura Model-View-Controller (MVC), adaptado para a plataforma JavaFX. Esse padrão divide a aplicação em três componentes principais, promovendo a separação de responsabilidades (Separation of Concerns).

Model (Modelo): Representa os dados e a lógica de negócio da aplicação. No nosso projeto, esta camada é distribuída em três pacotes:

- **org.example.model:** Contém as classes de dados puras (POJOs/Records) que representam as entidades do sistema, como Propriedade e UsuarioBiometria.
- **org.example.dao:** A Camada de Acesso a Dados (DAO), responsável por todo o código SQL e comunicação com o banco de dados.
- **org.example.service:** A Camada de Serviço, onde reside a lógica de negócio complexa, como o processamento de imagem e a lógica de comparação biométrica.

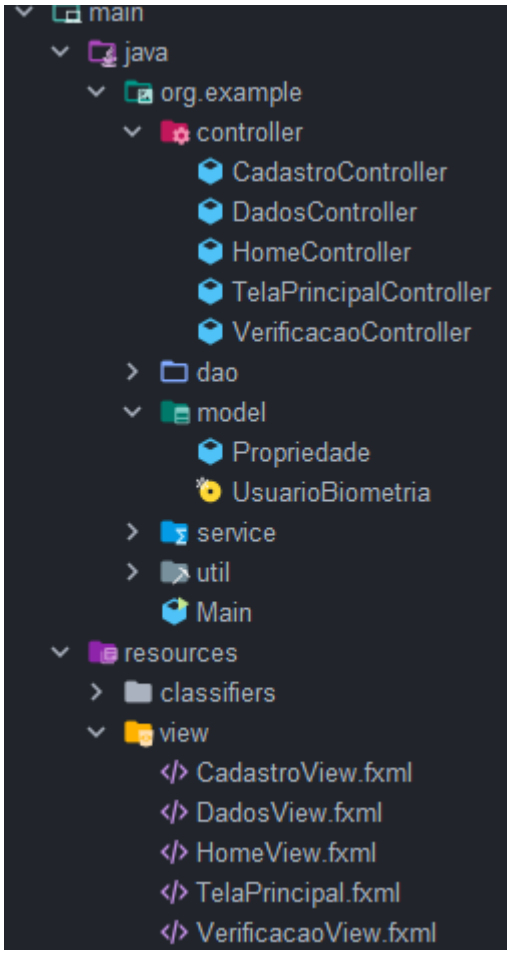
View (Visão): É a representação visual da aplicação, a interface com o usuário (UI). No nosso projeto, a View é composta exclusivamente pelos arquivos FXML localizados em src/main/resources/view/.

Controller (Controlador): É a camada que atua como intermediária entre a View e o Model. As classes no pacote org.example.controller ouvem as ações do usuário na View (ex: clique de botão), chamam os serviços e DAOs (Model) para processar a lógica e, em seguida, atualizam a View com o resultado.

Esta arquitetura torna o projeto escalável. Adicionar um novo tipo de biometria (ex: íris) exigiria apenas adicionar um novo método ao BiometriaService e ao UsuarioDAO, sem precisar alterar as telas existentes de forma significativa.

5.2. Estrutura Modular do Programa

O código-fonte foi organizado em pacotes Java, onde cada pacote representa um módulo com uma responsabilidade única:



org.example (Módulo Raiz)

●Main.java: A classe principal que inicializa a aplicação. Sua função é chamar o DatabaseService para garantir que o banco de dados esteja pronto e carregar a TelaPrincipal.fxml, que é a janela-mãe da aplicação.

org.example.controller (Módulo Controlador) Este módulo faz a ponte entre a interface do usuário (View) e a lógica de negócios (Model).

- TelaPrincipalController.java: Controla a "concha" (BorderPane). Sua única responsabilidade é carregar a HomeView.fxml no centro da tela assim que a aplicação é iniciada.
- HomeController.java: Controla a tela inicial. Seus métodos (irParaCadastro, irParaVerificacao) são acionados pelos botões da HomeView e usam o FXMLLoader para carregar as telas de cadastro ou verificação no BorderPane principal.
- CadastroController.java: É o controlador mais complexo. Ele gerencia a validação dos campos de texto, a lógica de exibição condicional dos campos de biometria (baseado na ComboBox), a inicialização do stream da webcam (chamando biometriaService.iniciarStreamWebcam), a captura do frame (biometriaService.capturarFrameEGravar) e a orquestração final do cadastro, chamando os serviços de extração de recursos e o DAO para salvar no banco.
- VerificacaoController.java: Espelha a lógica do CadastroController, mas para o fluxo de autenticação. Ele gerencia a seleção do tipo de verificação, captura a biometria necessária e coordena o UsuarioDAO (para buscar o template salvo) e o BiometriaService (para comparar os templates). Se a comparação for bem-sucedida, ele é responsável por carregar a DadosView.
- DadosController.java: Responsável por popular a TableView com os dados sigilosos. Possui um método público (carregarDados) que é chamado pelo VerificacaoController após o login, passando o nível de acesso do usuário. Este método então filtra e exibe os dados apropriados.

org.example.service (Módulo de Serviço)

- BiometriaService.java: O coração lógico do projeto. Esta classe encapsula toda a complexidade da visão computacional. Nenhum outro módulo (incluindo os Controllers) "sabe" como o OpenCV funciona; eles apenas chamam métodos de

alto nível como extrairRecursosRosto() ou compararBiometria(). Este isolamento é crucial para a manutenção.

org.example.dao (Módulo de Acesso a Dados)

- DatabaseService.java: Classe utilitária com métodos estáticos. É responsável por estabelecer a conexão JDBC com o arquivo SQLite e por executar o CREATE TABLE inicial, incluindo a inserção dos dados fictícios (popularDadosIniciais).
- UsuarioDAO.java: Implementa o padrão DAO (Data Access Object) para a entidade Usuario e Propriedade. Esta classe centraliza todo o código SQL (INSERT, SELECT) do sistema. Os Controllers não escrevem SQL; eles chamam métodos como usuarioDAO.cadastrarUsuario(...). É aqui que a regra de negócio de filtragem de dados (WHERE nivel_acesso_necessario <= ?) é implementada.

org.example.model (Módulo de Dados)

- UsuarioBiometria.java: Um record Java (uma classe de dados imutável) usado para transportar os dados biométricos lidos do banco (Mat da digital, Mat do rosto, nível de acesso) entre o UsuarioDAO e o VerificacaoController.
- Propriedade.java: Uma classe modelo que encapsula os dados da tabela propriedades_rurais. Ela utiliza JavaFX Properties (ex: SimpleStringProperty) em vez de String pura. Isso permite que a TableView na DadosView se vincule (bind) diretamente a esses atributos, atualizando a UI automaticamente.

org.example.util (Módulo Utilitário)

- AlertUtils.java: Uma classe auxiliar simples com métodos estáticos para padronizar a exibição de pop-ups de alerta, reduzindo a duplicação de código nos Controllers.

5.3. Fluxo de Dados e Casos de Uso

A interação entre os módulos da aplicação é melhor compreendida através dos seus dois principais casos de uso: o cadastro e a verificação.

O fluxo de **cadastro de usuário** inicia-se na HomeView, onde o usuário opta por se cadastrar. O HomeController carrega a tela CadastroView.fxml. O CadastroController assume o controle, e caso o usuário selecione "Rosto", ele exibe os controles da webcam. O CadastroController delega ao BiometriaService a tarefa de iniciar o stream da câmera e, ao clique do usuário, capturar e salvar um arquivo de imagem temporário. Ao submeter o formulário, o CadastroController envia este arquivo ao BiometriaService para processamento. O serviço primeiro utiliza um CascadeClassifier (Haar Cascade) para detectar e recortar o rosto da imagem e, em seguida, usa o algoritmo ORB para extrair o "template" biométrico (descritores). Este template (Mat) é então repassado ao UsuarioDAO, que, por sua vez, utiliza um método utilitário do BiometriaService para converter o Mat em um byte[]. Finalmente, o DAO executa um comando INSERT no banco de dados SQLite, armazenando o byte[] na coluna BLOB correspondente (face_dados).

O fluxo de **verificação de acesso** (Login) segue uma lógica complementar. O usuário navega para a VerificacaoView e informa seu nome de usuário e o método de autenticação (ex: "Rosto"). Após capturar uma nova imagem facial, o VerificacaoController orquestra a autenticação em etapas. Primeiro, ele solicita ao UsuarioDAO que busque o usuário. O DAO executa um SELECT, lê o BLOB e os metadados do banco e utiliza o BiometriaService para reconstruir o Mat do template facial armazenado. Esse template é retornado ao controller junto com o nível de acesso do usuário (ex: Nível 3). Em seguida, o controller envia a *nova* imagem capturada para o BiometriaService processar e gerar um *novo* template. Com os dois templates em mãos (o salvo e o novo), o controller chama o biometriaService.compararBiometria(). O serviço utiliza seu BFMatcher e o Teste de Razão (Lowe's Ratio Test) para verificar a correspondência. Se o resultado for true, o VerificacaoController carrega a DadosView.fxml e invoca o DadosController, passando o nível de acesso "3". Este último controller, então, solicita ao UsuarioDAO os dados das propriedades, que executa a consulta SQL final (...WHERE nivel_acesso_necessario <= 3), exibindo todos os dados na TableView e cumprindo assim os requisitos de segurança do projeto.

6. RELATÓRIO COM AS LINHAS DE CÓDIGO DO PROGRAMA

Este capítulo apresenta e analisa os trechos de código mais significativos desenvolvidos para o Sistema de Identificação e Autenticação Biométrica. O foco não é exibir todo o código-fonte, mas sim destacar as seções que implementam a lógica central do projeto, explicando sua função e como elas se conectam para atender aos requisitos da Atividade Prática Supervisionada.

O projeto foi estruturado seguindo o padrão Model-View-Controller (MVC), com camadas de serviço (Service) e de acesso a dados (DAO) bem definidas.

6.1. Definição do Banco de Dados (DatabaseService.java)

O coração do sistema é o banco de dados. O código abaixo, executado na inicialização, cria as tabelas usuarios e propriedades_rurais.

6.2. Detecção de Rosto (BiometriaService.java)

Para a biometria facial, não basta extrair recursos da foto inteira; precisamos primeiro *encontrar* o rosto. Este método usa o classificador Haar Cascade para isso.

```java
public Mat extrairRecursosRosto(File arquivoImagem) {  2 usages  SaadZ3
    Mat imagem = imread(arquivoImagem.getAbsolutePath(), IMREAD_GRAYSCALE);
    if (imagem.empty()) {
        throw new RuntimeException("Não foi possivel carregar a imagem do rosto.");
    }

    RectVector rostosDetectados = new RectVector();
    faceDetector.detectMultiScale(imagem, rostosDetectados);

    if (rostosDetectados.size() == 0) {
        System.err.println("Nenhum rosto detectado na imagem.");
        return new Mat();
    }

    Rect dadosRosto = rostosDetectados.get(0);
    Mat rostoRecortado = new Mat(imagem, dadosRosto);
    resize(rostoRecortado, rostoRecortado, new Size( _width: 150,  _height: 150));

    KeyPointVector keypoints = new KeyPointVector();
    Mat descritores = new Mat();
    orb.detectAndCompute(rostoRecortado, new Mat(), keypoints, descritores);
    return descritores;
}
```

Este é o núcleo do Processamento de Imagem e Visão Computacional. O código localiza, recorta e normaliza o rosto antes de extrair os descritores. Isso garante que a comparação seja feita apenas na área de interesse.

6.3. Comparação Biométrica (BiometriaService.java)

Após extrair os recursos, precisamos de um método confiável para compará-los. Usamos o "Lowe's Ratio Test" para garantir uma correspondência de alta qualidade.

```java
public boolean compararBiometria(Mat descritores1, Mat descritores2) {  2 usages  SaadZ3
    if (descritores1 == null || descritores2 == null || descritores1.empty() || descritores2.empty()) {
        return false;
    }

    DMatchVectorVector matches = new DMatchVectorVector();
    if(descritores1.cols() != descritores2.cols()) {
        System.err.println("Descritores incompatíveis.");
        return false;
    }

    matcher.knnMatch(descritores1, descritores2, matches, k: 2);

    int boasCorrelacoes = 0;
    for (long i = 0; i < matches.size(); i++) {
        DMatchVector match = matches.get(i);
        if (match.size() > 1) {
            DMatch m = match.get(0);
            DMatch n = match.get(1);
            if (m.distance() < RATIO_THRESH * n.distance()) {
                boasCorrelacoes++;
            }
        }
    }
    return boasCorrelacoes >= LIMIAR_MINIMO_PONTOS;
}
```

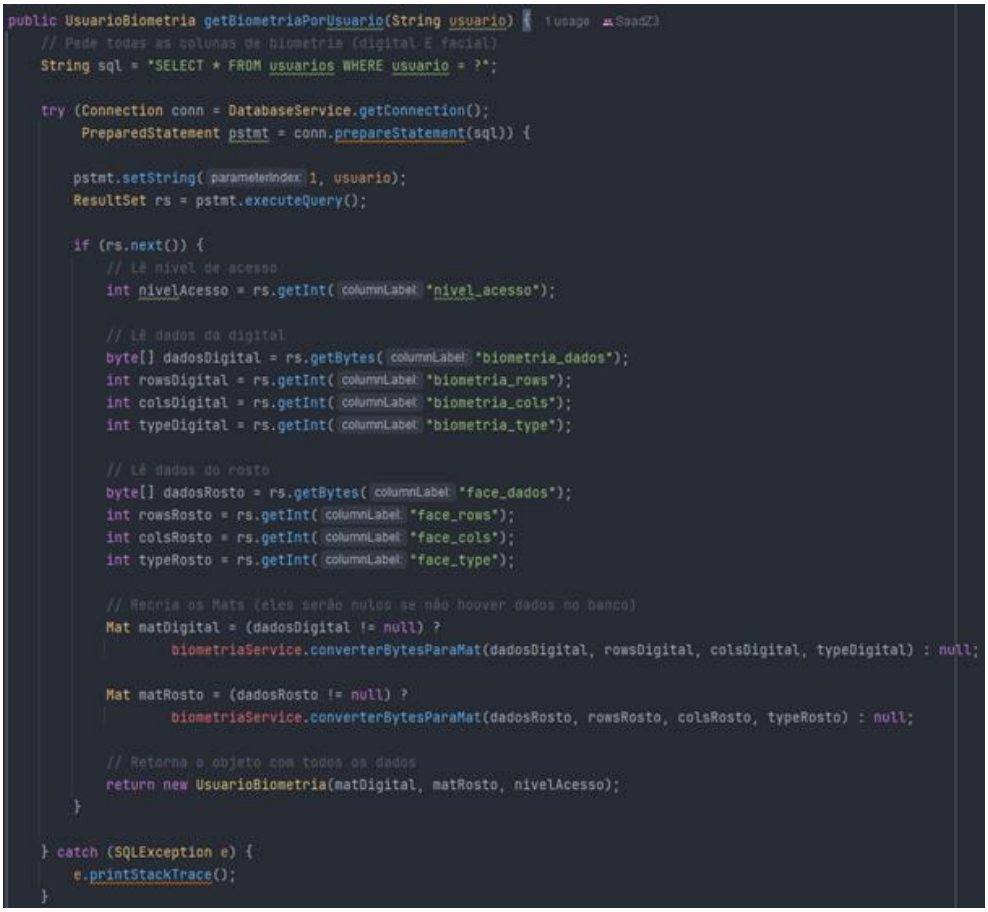
Em vez de aceitar qualquer correspondência, o for loop implementa um filtro. Ele só conta uma correspondência como "boa" se ela for significativamente melhor que a segunda opção. Isso elimina falsos positivos e torna o login seguro.

6.4. Integração com a Webcam (BiometriaService.java)

Para a captura facial, integramos a câmera diretamente na interface JavaFX. Isso requer o uso de *Threads* para não travar a aplicação.

6.5. Aplicando a Regra de Acesso (Login)

Após a autenticação, este método no UsuarioDAO é o que garante que o usuário só veja os dados do seu nível.



```
public UsuarioBiometria getBiometriaPorUsuario(String usuario) { 1 usage  SaadZ3
    // Pede todas as colunas de biometria (digital E facial)
    String sql = "SELECT * FROM usuarios WHERE usuario = ?";

    try (Connection conn = DatabaseService.getConnection();
         PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setString( parameterIndex: 1, usuario);
        ResultSet rs = pstmt.executeQuery();

        if (rs.next()) {
            // Lê nível de acesso
            int nivelAcesso = rs.getInt( columnLabel: "nivel_acesso");

            // Lê dados da digital
            byte[] dadosDigital = rs.getBytes( columnLabel: "biometria_dados");
            int rowsDigital = rs.getInt( columnLabel: "biometria_rows");
            int colsDigital = rs.getInt( columnLabel: "biometria_cols");
            int typeDigital = rs.getInt( columnLabel: "biometria_type");

            // Lê dados do rosto
            byte[] dadosRosto = rs.getBytes( columnLabel: "face_dados");
            int rowsRosto = rs.getInt( columnLabel: "face_rows");
            int colsRosto = rs.getInt( columnLabel: "face_cols");
            int typeRosto = rs.getInt( columnLabel: "face_type");

            // Recria os Mats (eles serão nulos se não houver dados no banco)
            Mat matDigital = (dadosDigital != null) ?
                    biometriaService.converterBytesParaMat(dadosDigital, rowsDigital, colsDigital, typeDigital) : null;

            Mat matRosto = (dadosRosto != null) ?
                    biometriaService.converterBytesParaMat(dadosRosto, rowsRosto, colsRosto, typeRosto) : null;

            // Retorna o objeto com todos os dados
            return new UsuarioBiometria(matDigital, matRosto, nivelAcesso);
        }

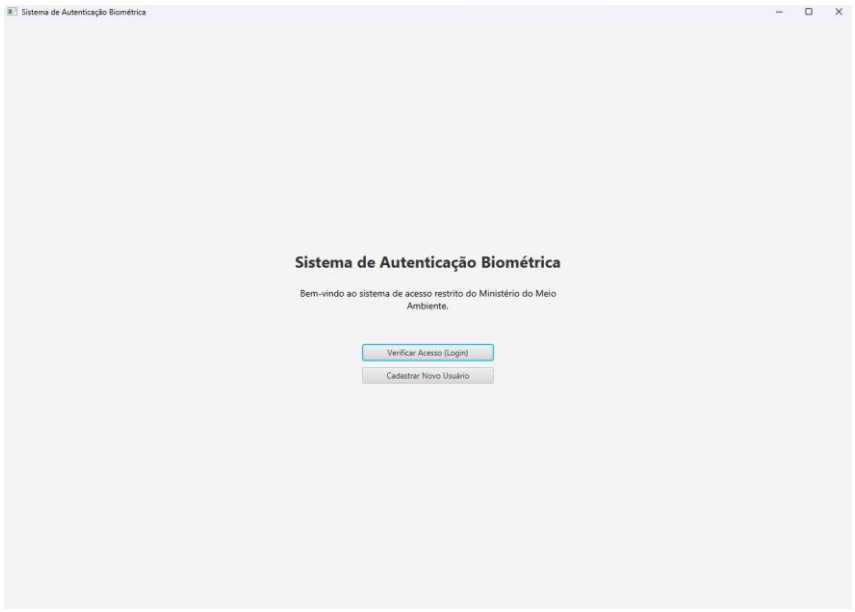
    } catch (SQLException e) {
        e.printStackTrace();
    }
```

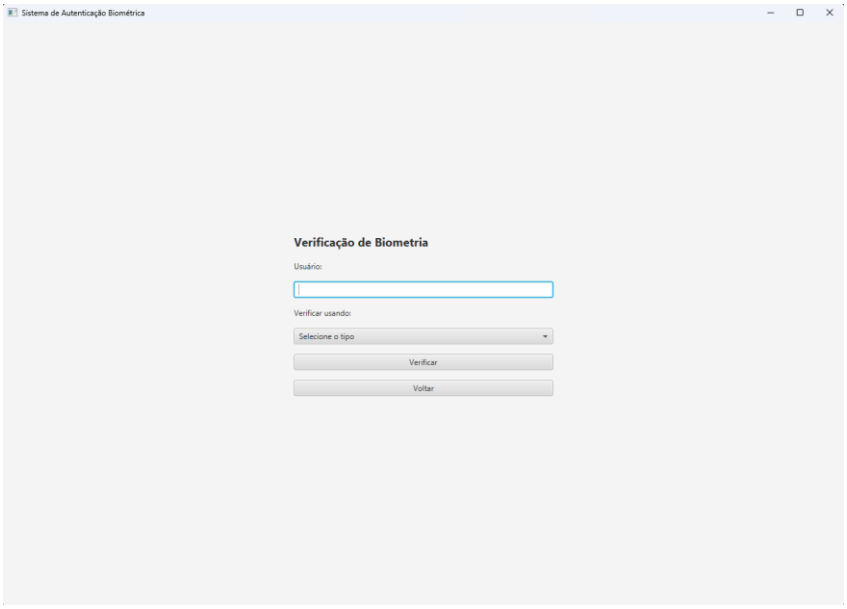
Esta é a implementação direta da regra de negócio. A consulta WHERE nivel_acesso_necessario <= ? é a parte mais importante. Se o usuário logado tem nivelUsuario = 2, a consulta retorna todos os dados de nível 1 E 2. Se ele tem nível 3, retorna 1, 2 E 3. Isso garante a hierarquia de acesso solicitada.

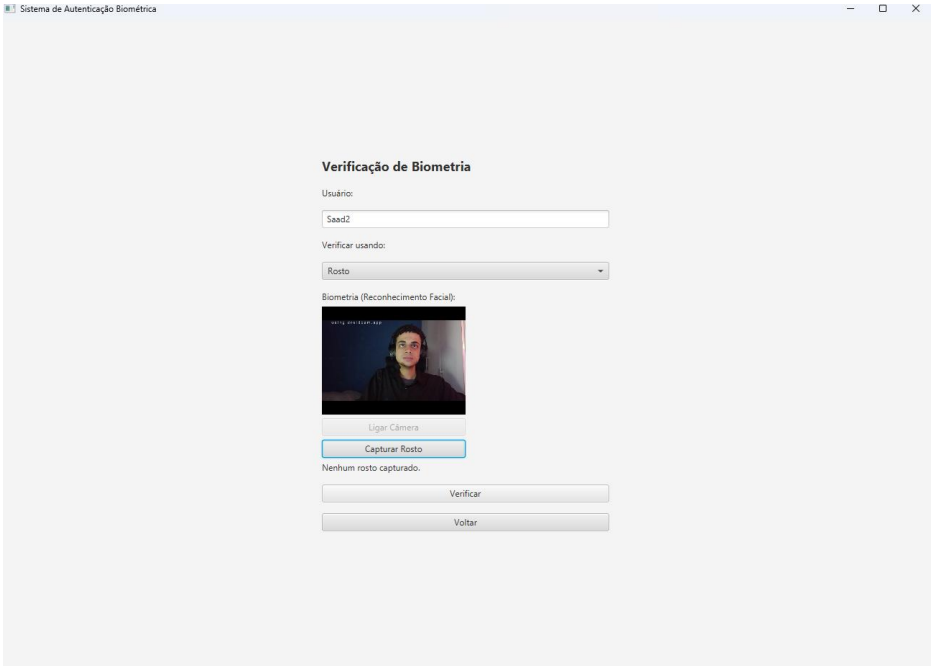


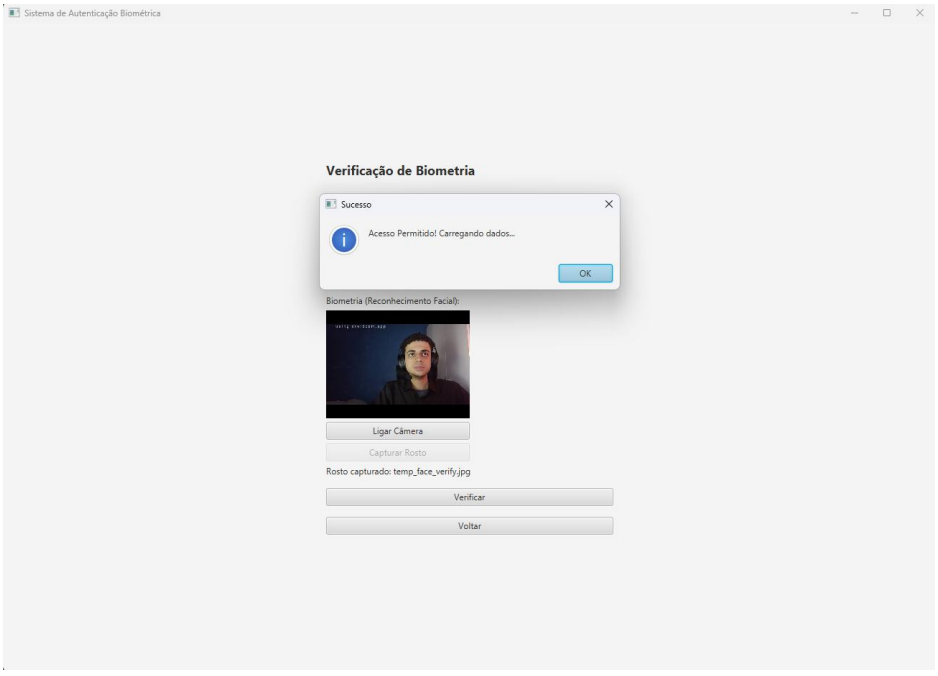
```
public void iniciarStreamWebcam(ImageView view) { 2 usages  SaadZ3
    stopFlag = false;
    grabber = new OpenCVFrameGrabber( deviceNumber: 0);
    threadWebcam = new Thread(() -> {
        try {
            grabber.start();
            Thread.sleep( millis: 200); // Warm-up
            while (!stopFlag) {
                Frame frame = grabber.grab();
                if (frame == null) break;
                frameAtual = matConverter.convert(frame);
                Image fxImage = fxConverter.convert(frame);
                Platform.runLater(() -> view.setImage(fxImage));
            }
        } catch (Exception e) { e.printStackTrace();
        } finally {
            try { grabber.stop(); grabber.release(); } catch (Exception e) { e.printStackTrace(); }
        }
    });
    threadWebcam.setDaemon(true);
    threadWebcam.start();
}
```

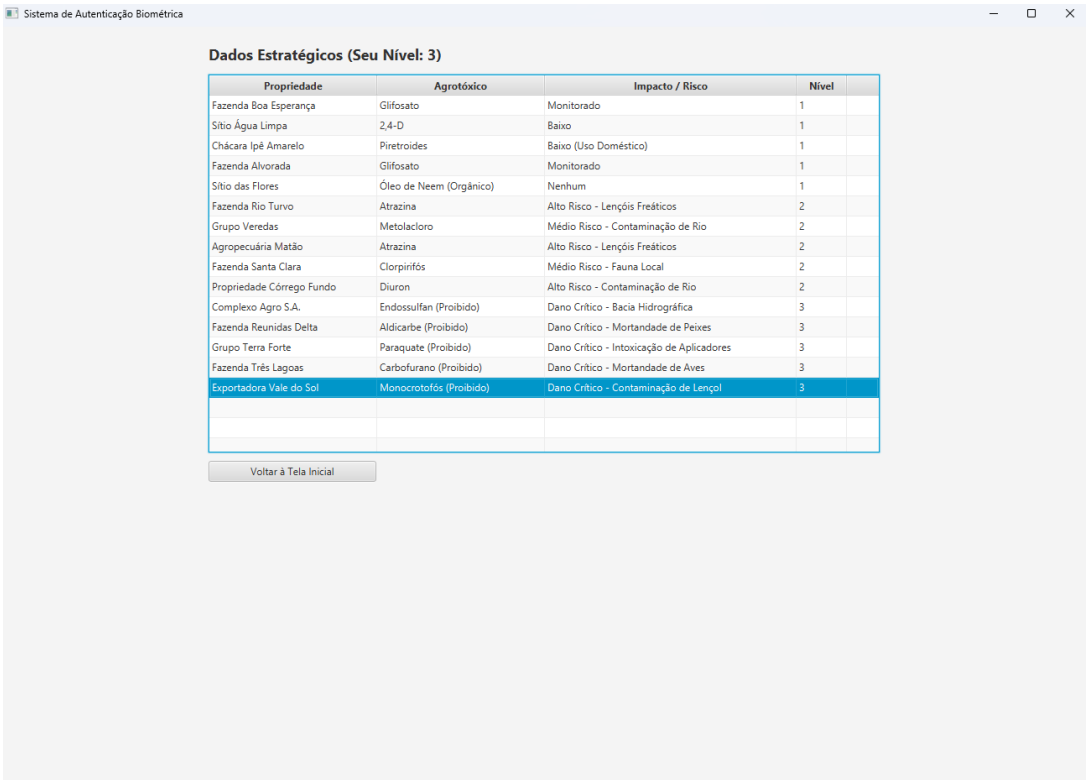
7. APRESENTAÇÃO DO PROGRAMA EM FUNCIONAMENTO






Sistema de Autenticação Biométrica
Verificação de Biometria
Usuário:
Saad2
Verificar usando:
Rosto
Biometria (Reconhecimento Facial):
Ligar Câmera
Capturar Rosto
Nenhum rosto capturado.
Verificar
Voltar


Sistema de Autenticação Biométrica
Verificação de Biometria
Sucesso
Acesso Permitido! Carregando dados...
OK
Biometria (Reconhecimento Facial):
Ligar Câmera
Capturar Rosto
Rosto capturado: temp_face_verify.jpg
Verificar
Voltar


Sistema de Autenticação Biométrica
Dados Estratégicos (Seu Nível: 3)
Propriedade
Agrotóxico
Impacto / Risco
Nível
Fazenda Boa Esperança
Glifosato
Monitorado
1
Sítio Água Limpa
2,4-D
Baixo
1
Chácara Ipê Amarelo
Piretroides
Baixo (Uso Doméstico)
1
Fazenda Alvorada
Glifosato
Monitorado
1
Sítio das Flores
Óleo de Neem (Orgânico)
Nenhum
1
Fazenda Rio Turvo
Atrazina
Alto Risco - Lençóis Freáticos
2
Grupo Veredas
Metolacloro
Médio Risco - Contaminação de Rio
2
Agropecuária Matão
Atrazina
Alto Risco - Lençóis Freáticos
2
Fazenda Santa Clara
Clorpirifós
Médio Risco - Fauna Local
2
Propriedade Córrego Fundo
Diuron
Alto Risco - Contaminação de Rio
2
Complexo Agro S.A.
Endossulfan (Proibido)
Dano Crítico - Bacia Hidrográfica
3
Fazenda Reunidas Delta
Aldicarbe (Proibido)
Dano Crítico - Mortandade de Peixes
3
Grupo Terra Forte
Paraquate (Proibido)
Dano Crítico - Intoxicação de Aplicadores
3
Fazenda Três Lagoas
Carbofurano (Proibido)
Dano Crítico - Mortandade de Aves
3
Exportadora Vale do Sol
Monocrotofós (Proibido)
Dano Crítico - Contaminação de Lençol
3
Voltar à Tela Inicial

8. BIBLIOGRAFIA

ASIMOV ACADEMY. O que é OpenCV e para que serve?. 23 set. 2024. Disponível em: https://hub.asimov.academy/blog/o-que-é-opencv-e-para-que-serve. Acesso em: 5 nov. 2025.

BRY. O que é Biometria. In: **Blog BRY**. São Paulo, 28 set. 2022. Disponível em: https://www.bry.com.br/blog/o-que-e-biometria/. Acesso em: 6 nov. 2025.

EMARA, Taha. Real Time Face Detection using OpenCV with Java "with code". [S.l.]: YouTube, 3 jan. 2015. 1 vídeo (14 min, 4 seg). Disponível em: https://www.youtube.com/watch?v=WeLET1tZPaE. Acesso em: 6 nov. 2025.

GOOGLE. Gemini. Disponível em: https://gemini.google.com/app?hl=pt-BR. Acesso em: 10 nov. 2025.

HID GLOBAL. Biometria de Impressão Digital. [S.l.: s.n.], [s.d.]. Disponível em: https://www.hidglobal.com/pt/solutions/fingerprint-biometrics. Acesso em: 6 nov. 2025.

IRISGUARD. How it works (Como funciona). [S.l.]: [s.n.], [s.d.]. Disponível em: https://www-irisguard-com.translate.goog/technology/how-it-works/?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=sge. Acesso em: 7 nov. 2025.

KASPERSKY. Biometria. [S.l.: s.n.], [s.d.]. Disponível em: https://www.kaspersky.com.br/resource-center/definitions/biometrics. Acesso em: 7 nov. 2025.

MOUKTHIKA. Fingerprint Matching Using OpenCV. In: **OpenCV Blog**. 10 abr. 2025. Disponível em: https://opencv.org/blog/fingerprint-matching-using-opencv/. Acesso em: 8 nov. 2025.

OPENCV. Feature Matching (FLANN). In: **OpenCV Docs**. Disponível em: https://docs.opencv.org/3.4/d5/d6f/tutorial_feature_flann_matcher.html. Acesso em: 5 nov. 2025.

OPENCV. OpenCV. Disponível em: https://opencv.org. Acesso em: 5 nov. 2025.

OPENCV. Tutorial: Face Detection with JavaScript. In: **OpenCV Docs**. Disponível em: https://docs.opencv.org/4.x/d2/d99/tutorial_js_face_detection.html. Acesso em: 5 nov. 2025.

PROCESSAMENTO de Imagem e Visão Computacional (30hs - CCOMP). **Material didático**. [S.l.]: [s.n.], [s.d.]. 1 PDF.

PROGRAMMINGKNOWLEDGE. Java prog#149. Capture Image from webcam using JavaCV and Java. [S.l.]: YouTube, 27 nov. 2013. 1 vídeo (6 min, 15 seg). Disponível em: http://www.youtube.com/watch?v=Qpt-CUuTJkY. Acesso em: 8 nov. 2025.

SQLITETUTORIAL.NET. SQLite Java: Write and Read BLOB. In: **SQLite Tutorial**. Disponível em: https://www.sqlitetutorial.net/sqlite-java/jdbc-read-write-blob. Acesso em: 7 nov. 2025.

THE DEEP HUB. Detecting and Tracking Objects with ORB using OpenCV. In: **Medium**. Disponível em: https://medium.com/thedeephub/detecting-and-tracking-objects-with-orb-using-opencv-d228f4c9054e. Acesso em: 9 nov. 2025.

VALID CERTIFICADORA. Biometria da íris no Brasil: revolução digital ou risco para sua privacidade? In: **Blog Valid Certificadora**. [S.l.: s.n.], [s.d.]. Disponível em: https://validcertificadora.com.br/blogs/certificado-digital/biometria-da-iris-no-brasil-revolucao-digital-ou-risco-para-sua-privacidade. Acesso em: 10 nov. 2025.

9. FICHA ATIVIDADES PRÁTICAS SUPERVISIONADAS